



Zulemarodriguez438@gmail.com
Alvarez.maximilianoaa@gmail.com

Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

INSTITUCIÓN: Universidad Tecnológica Nacional (UTN)

CARRERA: Tecnicatura Universitaria en Programación

MATERIA: Programacion 1

TÍTULO DEL PROYECTO: Gestión de Datos de Países en Python
ESTUDIANTES:

Zulema Rodríguez — zulemarodriguez438@gmail.com

Maximiliano Álvarez — alvarez.maximilianoaa@gmail.com

FECHA DE ENTREGA:17/06/2026

ÍNDICE

Introducción____3

Diagramas de flujo____3

Decisiones Técnicas y Arquitectura

2.1. Explicación breve de cada función____5

Marco Teórico

3.1. Estructuras de datos en memoria____6

3.2. Normalización Unicode____6

3.3. Validación de entrada____6

3.4. Algoritmo de ordenamiento manual____6

Cómo se manejaron los errores____6

Dificultades____7

Conclusión____7

Repositorio y Material Complementario____8

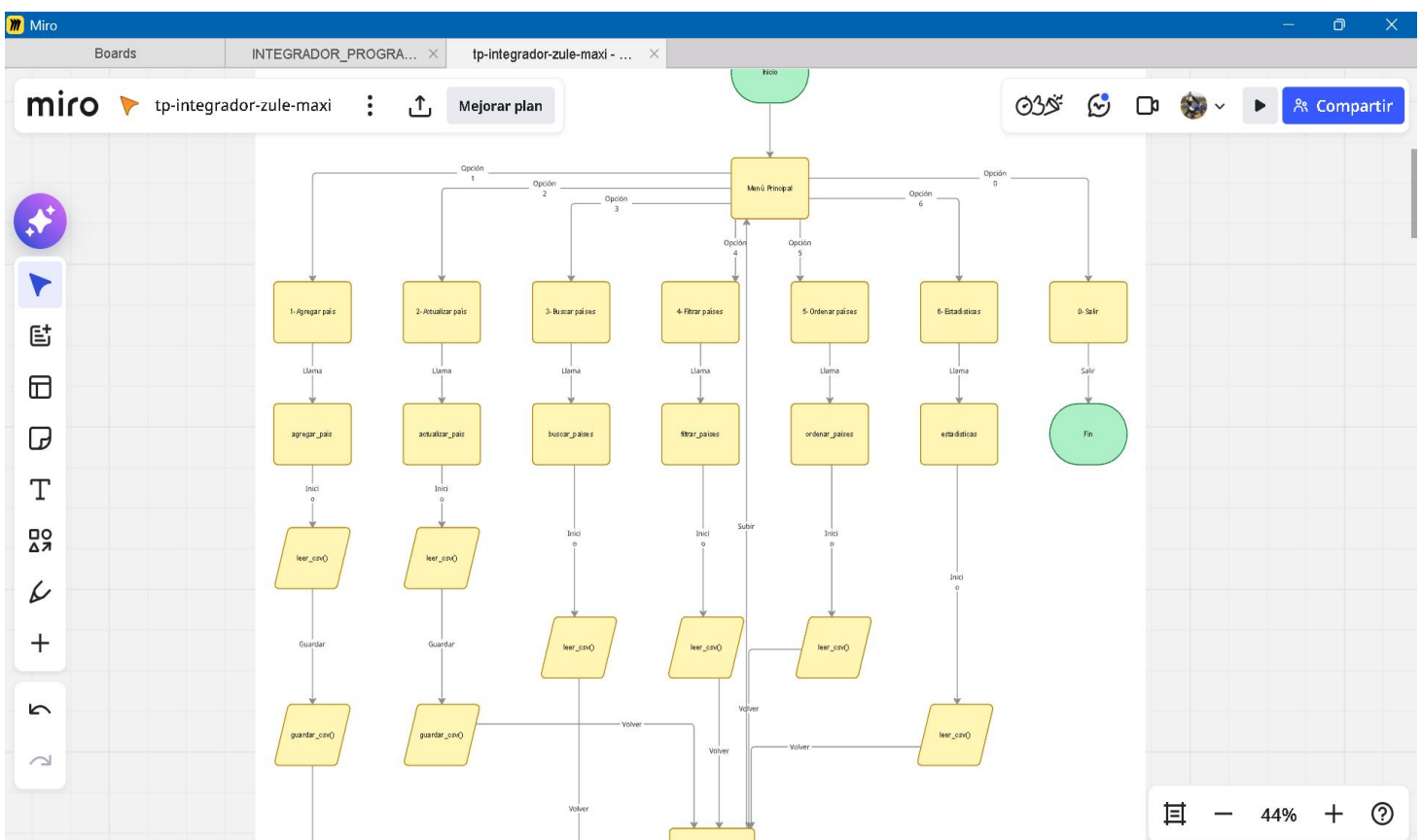
Introducción:

Este informe presenta el desarrollo de un sistema de gestión de países en Python, que permite agregar, actualizar, buscar, filtrar, ordenar y obtener estadísticas sobre datos geográficos (nombre, población, superficie y continente), persistiendo la información en un archivo CSV.

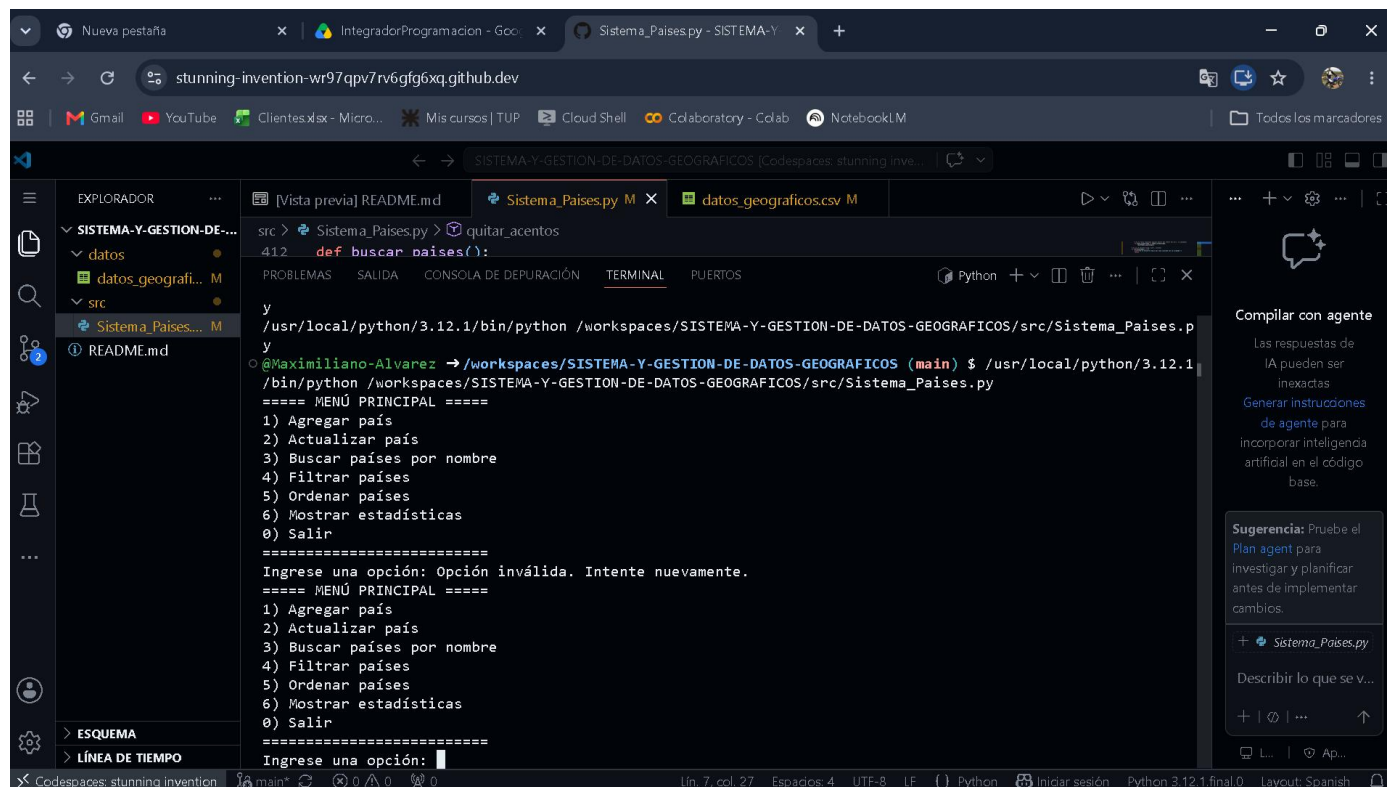
El trabajo aborda dificultades propias de este tipo de aplicaciones, como la validación de datos ingresados por el usuario, el manejo de texto con acentos al buscar y comparar países, y la necesidad de que el programa siga funcionando ante errores inesperados.

El programa está desarrollado bajo el paradigma de la programación estructurada/procedural en Python, utilizando como estructuras de datos principales listas y diccionarios para representar y manipular la información de los países.

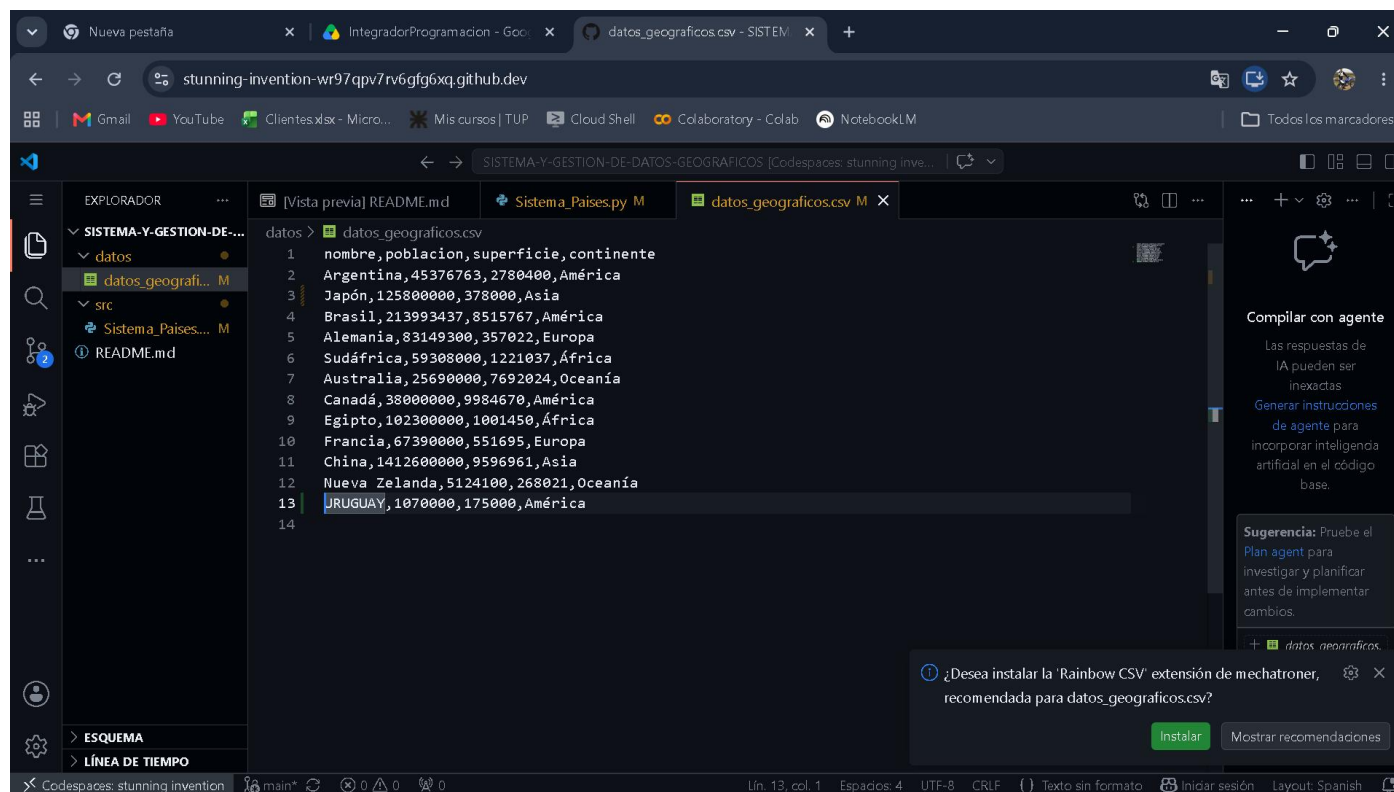
Diagrama de Flujo:



Menu principal en Codespaces:



Archivo csv: Datos_geograficos.csv



Decisiones Técnicas y Arquitectura:

Explicación breve de cada función:

`quitar_acentos(texto)` — Recibe un string y devuelve el mismo texto sin tildes, usando descomposición Unicode NFD y filtrando los caracteres marcados como diacríticos.

`leer_csv()` — Abre el archivo CSV, convierte cada fila en un diccionario con los tipos correctos (población y superficie como int), descarta filas inválidas y devuelve la lista completa de países. Si el archivo no existe, devuelve lista vacía sin romper el programa.

`guardar_csv(lista_paises)` — Crea la carpeta datos si no existe y reescribe el archivo CSV completo a partir de la lista de diccionarios recibida.

`elegir_continente()` — Muestra un menú numerado de continentes válidos y obliga al usuario a elegir una opción dentro del rango, devolviendo el nombre del continente seleccionado.

`agregar_pais()` — Pide nombre, población, superficie y continente; valida que el nombre no esté vacío ni contenga números, chequea que no exista ya un país con ese nombre (comparación normalizada) y, si todo es válido, agrega el país y guarda el CSV.

`filtrar_paises()` — Ofrece filtrar por continente (usando `elegir_continente` y comparación normalizada) o por rango de población (pidiendo mínimo y máximo), mostrando los países que cumplen la condición.

`estadisticas()` — Recorre todos los países calculando el de mayor y menor población, el promedio de población y superficie, y la cantidad de países por continente usando un diccionario contador.

`actualizar_pais()` — Busca un país por nombre (normalizado), muestra sus datos actuales y permite modificar nombre, población, superficie o continente de forma individual, guardando los cambios al final.

`buscar_paises()` — Busca países cuyo nombre contenga (coincidencia parcial) el texto ingresado, normalizando tanto la búsqueda como los nombres almacenados.

`ordenar_paises()` — Ordena la lista de países por nombre, población o superficie usando un algoritmo de burbuja implementado manualmente, y muestra el resultado ordenado.

Bucle principal (menú) — Muestra las opciones disponibles, lee la elección del usuario y llama a la función correspondiente; controla la salida del programa y captura interrupciones de teclado.

Selección (estructuras condicionales): uso de `if / elif / else` para tomar decisiones, como en el menú principal o al elegir qué dato actualizar en `actualizar_pais`.

Iteración (bucles): uso de `while` para validar entradas repetidamente hasta que sean correctas, y de `for` para recorrer listas de países (por ejemplo, al buscar, filtrar u ordenar).

Modularización por funciones: cada tarea del sistema está encapsulada en una función con un propósito específico (`leer`, `guardar`, `buscar`, `ordenar`, etc.), lo que favorece la reutilización de código (por ejemplo, `leer_csv` y `quitar_acentos` se usan desde varias funciones) y facilita la lectura y el mantenimiento del programa.

Estructuras de datos básicas: uso de listas para representar colecciones (la lista de países) y diccionarios para representar registros con campos nombrados (cada país), en lugar de clases u objetos.

Marco Teórico:

Desde el punto de vista teórico, se apoya en conceptos de persistencia de datos mediante archivos CSV (a través del módulo csv), validación de entradas de usuario y manejo de excepciones (try/except) para garantizar la robustez frente a errores.

El menú principal actúa como controlador central, coordinando la interacción con el usuario y derivando cada opción elegida a la función correspondiente.

Este diseño separa claramente el almacenamiento de los datos, el procesamiento de la información y la interfaz de usuario (basada en consola), lo que facilita el mantenimiento y la futura incorporación de las funciones aún pendientes (actualizar, buscar y ordenar países).

Estructuras de datos en memoria: cada país se representa como un diccionario, y el conjunto de países como una lista de diccionarios. Es la estructura típica para trabajar registros tabulares sin usar pandas ni bases de datos.

Normalización Unicode: se usa `unicodedata.normalize('NFD', texto)` para separar las letras de sus marcas diacríticas (tildes) y luego filtrar esas marcas (`category(c) != 'Mn'`). Esto resuelve el problema de comparar strings que deberían ser equivalentes para el usuario ("Japón" vs "Japon") pero que en Python son distintos si se comparan literalmente.

Validación de entrada (defensive input): bucles while que no dejan avanzar el flujo hasta que el dato ingresado cumple el formato esperado (no vacío, solo letras, numérico, dentro de un rango de opciones), evitando que datos corruptos lleguen a la lógica de negocio.

Algoritmo de ordenamiento manual: se implementa ordenamiento por burbuja dado en clase en lugar de usar `sorted()`.

Cómo se manejaron los errores:

Errores de archivo (leer_csv/guardar_csv): `FileNotFoundError` se captura específicamente para inicializar con lista vacía cuando el CSV todavía no existe; un `except Exception` genérico cubre otros problemas de I/O (permisos, disco lleno, etc.) sin interrumpir el programa.

Errores de datos fila por fila: dentro de leer_csv, cada fila se procesa en su propio try/except (`ValueError`, `KeyError`), de modo que una sola fila corrupta (con un campo no numérico o faltante) se descarta con un aviso, sin perder el resto del archivo.

Validación de entrada del usuario: en vez de capturar excepciones, se usan bucles while que no dejan avanzar hasta que el dato es válido (nombre no vacío, solo letras, número entero válido, opción de menú dentro de rango). Es validación preventiva más que manejo de excepciones, aunque en los campos numéricos sí se combina con try/except `ValueError` para capturar conversiones fallidas a int.

Errores a nivel de cada función de negocio: `agregar_pais`, `actualizar_pais`, `buscar_paises`, `filtrar_paises`, `ordenar_paises` y estadísticas envuelven toda su lógica en un try/except `Exception` general, de manera que cualquier error no previsto dentro de esa operación se informa por consola pero no interrumpe el programa ni afecta otras funciones.

Errores a nivel del bucle principal: el while True del menú tiene su propio try/except, distinguiendo (`KeyboardInterrupt`, `EOFError`) —para salir prolijamente si el usuario interrumpe con Ctrl+C o Ctrl+D— de cualquier otro `Exception`, que se reporta y permite que el menú vuelva a aparecer.

DIFICULTADES:

Persistencia sin base de datos: todo el estado vive en un archivo de texto. Esto obliga a leer el CSV completo al inicio de cada operación y reescribirlo completo al final (no hay actualización incremental), lo cual es simple pero ineficiente para volúmenes grandes y no es seguro ante escrituras concurrentes.

Tipos de datos en CSV: un CSV guarda todo como texto, así que población y superficie deben convertirse explícitamente a int al leer, y cualquier fila con datos faltantes o no numéricos puede romper el parseo (resuelto con try/except (ValueError, KeyError) fila por fila, descartando solo la fila problemática en vez de todo el archivo).

Comparación de texto con acentos y mayúsculas: buscar, filtrar o detectar duplicados de "México" debe funcionar aunque el usuario escriba "mexico" o "MÉXICO". Se resolvió normalizando (minúsculas + sin espacios + sin acentos) tanto el dato ingresado como el dato guardado antes de comparar.

Evitar duplicados: detectar si un país ya existe requiere la misma normalización, ya que comparar strings crudos generaría falsos negativos (no detectaría "Brasil" y "brasil" como el mismo país). Validación robusta de menús y datos numéricos: el usuario puede ingresar cualquier cosa por teclado, así que cada input numérico o de opción de menú está protegido con bucles de reintento.

Evitar que el programa se cierre por errores no previstos: dado que es una app de consola interactiva, un error sin capturar mataría el programa entero y el usuario perdería el menú principal; por eso cada función de negocio y el bucle del menú están envueltos en try/except genéricos.

CONCLUSION:

El desarrollo de este proyecto integrador nos permitió consolidar con éxito las bases teóricas y prácticas de la programación. A lo largo del diseño del sistema, el mayor desafío técnico consistió en pasar de una lógica de código lineal a una arquitectura modularizada. Esta organización mediante funciones independientes y especializadas no solo garantizó la claridad del flujo, sino que dota al software de escalabilidad, permitiendo futuras actualizaciones o el mantenimiento del código de manera sencilla y eficiente.

Uno de los pilares fundamentales del software es su robustez. Mediante la implementación de un control estricto de excepciones a través de bloques try-except, se logró blindar el sistema frente a posibles errores de tipo o rango en las entradas del usuario. Esto asegura la integridad de la información antes de ser procesada y un comportamiento resistente a fallos que evita el colapso de la aplicación.

Se destaca el diseño del algoritmo de ordenamiento por comparación e intercambio de posiciones para la manipulación dinámica del dataset, y la optimización de búsquedas parciales mediante operadores de pertenencia y procesos de normalización de caracteres. El flujo concluye de manera segura logrando la persistencia de los datos en archivos físicos CSV, garantizando que el estado de la información se sincronice y permanezca inalterable una vez finalizada la ejecución.

Consideramos que la capacidad de sintetizar problemas complejos de gestión de datos y traducirlos en estructuras lógicas eficientes y robustas constituye la competencia más valiosa, sentando bases sólidas para nuestro desarrollo como futuros profesionales de la tecnología.

REPOSITORIO: <https://github.com/Zulema-Rodriguez/SISTEMA-Y-GESTION-DE-DATOS-GEOGRAFICOS>

YOUTUBE: <https://www.youtube.com/watch?v=XokzrS1OL5o>